

Smart Pole — Data Push Integration Guide

คู่มือมาตรฐานสำหรับระบบผู้รับข้อมูล (Receiver) — ทุกสายที่จะเชื่อมต่อ

STANDARD v1.0 · One-way Secure Push

Document version 1.0 · schemaVersion 1.0 · Signature: Ed25519

ผู้จัดทำ: Smart Pole Platform · อัปเดต: 2026-08-05

เอกสารนี้เป็น **มาตรฐานกลาง** สำหรับระบบภายนอกที่ต้องการรับข้อมูล sensor (อุณหภูมิ / ความชื้น / PM2.5) จากแพลตฟอร์ม Smart Pole แบบ **push ทางเดียว (one-way)** ผู้เชื่อมต่อทุกสายใช้สเปคชุดเดียวกันนี้เพื่อความปลอดภัยและความสม่ำเสมอ

สารบัญ

- ภาพรวมและสถาปัตยกรรม (Overview)
- สิ่งที่ผู้รับต้องเตรียม (Prerequisites)
- รูปแบบข้อมูล (Payload)
- HTTP Headers
- ความปลอดภัยและการตรวจสอบ (Security & Verification) — สำคัญสุด
- สัญญาการตอบกลับ (HTTP Response Contract)
- การจัดเก็บและตรวจข้อมูล (Data Handling)
- การจัดการข้อมูลขาด (Gap Handling)
- การเสริมความปลอดภัย (Hardening — แนะนำ)
- สิ่งที่เราส่งมอบ / สิ่งที่คุณไม่ต้องทำ
- ภาคผนวก: ตัวอย่าง, Pseudo-code, Test cases

1. ภาพรวมและสถาปัตยกรรม

ทิศทางการสื่อสารเป็นแบบ **ทางเดียว** — แพลตฟอร์ม Smart Pole (ผู้ส่ง) เป็นฝ่าย **POST** ข้อมูลเข้าหา endpoint ของผู้รับ ผู้รับ **ไม่ต้อง** เรียกกลับหรือ poll ข้อมูล

[Smart Pole Host] —HTTPS POST (signed)—> [Receiver Endpoint]
ผู้ส่ง ผู้รับ (verify + store)

หัวข้อ	ค่า
Method	POST + JSON body
จังหวะส่ง	1 record ทุก 5 นาที ต่อเสา (snapshot ค่าล่าสุด — ไม่ใช่ค่าเฉลี่ย)
Authentication	Ed25519 digital signature (asymmetric — ผู้รับใช้ public key เท่านั้น)
Transport	HTTPS / TLS 1.2+
Delivery guarantee	At-least-once + idempotency (eventId)

2. สิ่งที่ต้องเตรียม (Prerequisites)

- 1. **ต้องมี** **HTTPS endpoint** + TLS certificate ที่ valid (TLS 1.2 ขึ้นไป)
- 2. **ต้องมี** รับจากผู้ส่ง 4 อย่าง: **Public key (Ed25519)**, **Key-Id**, เอกสารสเปคนี้, ตัวอย่าง signed request
- 3. **ต้องมี** Storage 2 ชุด: (ก) ตารางเก็บข้อมูล sensor (ข) ตาราง/cache เก็บ **eventId** ที่รับแล้ว (สำหรับ dedup)
- 4. **ต้องมี** ตกลงร่วมกัน: endpoint path, หน่วยวัด, interval, Key-Id ปัจจุบัน

3. รูปแบบข้อมูล (Payload)

```
{
  "schemaVersion": "1.0",
  "source": "smart-pole",
  "eventId": "9f1c2b7e-4d3a-4b6e-9c11-8a5e2f0d7b34",
  "sentAt": "2026-08-05T02:35:00.000Z",
  "poleName": "pole-01",
  "measuredAt": "2026-08-05T02:34:12.123Z",
  "seq": 12345,
  "metrics": {
    "temperature": 28.00,
    "humidity": 87.60,
    "pm25": 13.00
  }
}
```

Field	Type	รูปแบบ / หมายเหตุ
schemaVersion	string	เวอร์ชันโครงสร้าง (ปัจจุบัน "1.0")
source	string	ระบบต้นทาง "smart-pole"
eventId	string	UUID v4 — ใช้ dedup
sentAt	string	UTC ISO 8601 — เวลาที่ส่ง
poleName	string	รหัสเสา
measuredAt	string	UTC ISO 8601 — เวลาที่วัดจริง (ใช้เป็นเวลาของข้อมูล)
seq	number	ลำดับจาก firmware — ใช้ตรวจสอบข้อมูลซ้ำ/ขาด
metrics.temperature	number \ null	2 ทศนิยม
metrics.humidity	number \ null	2 ทศนิยม
metrics.pm25	number \ null	2 ทศนิยม

หน่วยวัด (ล็อกในสเปค — ไม่อยู่ใน payload): temperature = °C · humidity = %RH · pm25 = µg/m³
metrics แต่ละตัวเป็น null ได้ (กรณี sensor อ่านได้บางค่า) — ผู้รับต้องยอมรับ null

4. HTTP Headers

```
POST <endpoint> HTTP/1.1
Content-Type: application/json
X-SmartPole-Key-Id:      spole-2026
X-SmartPole-Timestamp:   1785807300
X-SmartPole-Signature:   ed25519=<base64 signature>
X-SmartPole-Event-Id:    9f1c2b7e-4d3a-4b6e-9c11-8a5e2f0d7b34
```

Header	ความหมาย
X-SmartPole-Key-Id	ระบุว่าใช้ public key ตัวไหน verify (รองรับการหมุน key)
X-SmartPole-Timestamp	Unix seconds ตอนส่ง — ใช้กัน replay
X-SmartPole-Signature	ลายเซ็น Ed25519 (base64) ของ timestamp + "." + rawBody
X-SmartPole-Event-Id	ตรงกับ eventId ใน body — ใช้ dedup

5. ความปลอดภัยและการตรวจสอบ (Security & Verification)

ส่วนสำคัญที่สุด — ทำ ตามลำดับ ก่อนรับข้อมูลเข้าระบบ

- 1 **TLS:** รับเฉพาะ HTTPS (ปฏิเสธ http)
- 2 อ่าน raw body (bytes) เก็บไว้ก่อน parse — signature คำนวณบน raw body ถ้า parse แล้ว serialize ใหม่จะไม่ตรง
- 3 **ตรวจสอบ Timestamp:** ถ้า |now - timestamp| > 300 วินาที (5 นาที) → ปฏิเสธ (401) [กัน replay]

4 เลือก public key ตาม X-SmartPole-Key-Id

5 Verify signature (Ed25519):

```
message = timestamp + "." + rawBody
signature = base64_decode( X-SmartPole-Signature ไม่รวม "ed25519=" )
valid = Ed25519_verify(publicKey, signature, message)
ถ้า valid = false → ปฏิเสธ (401 Unauthorized)
```

6 Dedup: ถ้า eventId เคยรับแล้ว → ตอบ 2xx ทันที (idempotent — ไม่ประมวลผล/นับซ้ำ)

สำคัญ: ต้อง verify signature ก่อน นำข้อมูลไปใช้เสมอ — signature ที่ผ่านยืนยันว่า (ก) มาจาก Smart Pole จริง (ข) payload ไม่ถูกแก้ไขระหว่างทาง

6. สัญญาการตอบกลับ (HTTP Response Contract)

ผู้รับตอบ	ความหมาย	ผู้ส่งจะ
2xx (200/204)	รับสำเร็จ (หรือ dedup แล้ว)	ไม่ retry
4xx (400/401/422)	คำขอผิด (signature/schema/timestamp ไม่ผ่าน)	ไม่ retry (ต้องแก้ที่ต้นเหตุ)
5xx (500/503)	ปัญหาชั่วคราวฝั่งผู้รับ	retry แบบ exponential backoff

ตอบให้เร็ว (ภายใน ~10 วินาที) — แนะนำ "รับแล้วตอบ 2xx ก่อน แล้วค่อยประมวลผล/เขียน DB แบบ async" เพื่อไม่ให้ผู้ส่ง retry เพราะผู้รับช้า

กรณี receiver รับไม่ไหว (overload): ให้ตอบ 5xx — อย่าปล่อย timeout · ผู้ส่งมี retry (จำกัดครั้ง) + circuit breaker ที่ จะ backoff แล้ว "หยุดยิงชั่วคราว" ให้เองเมื่อผู้รับส่ง 5xx ต่อเนื่อง → ช่วยลดภาระทั้งสองฝั่ง (ผู้ส่งไม่กระหน่ำ ผู้รับได้พัก)

7. การจัดเก็บและตรวจสอบข้อมูล (Data Handling)

- parse JSON → ตรวจสอบ field ครบ + type ถูกต้อง (schema validation)
- ตรวจสอบ schemaVersion ว่ารองรับ — ถ้าเวอร์ชันไม่รู้จักรับ ตอบ 4xx
- บันทึก eventId (สำหรับ dedup) พร้อมข้อมูล
- ใช้ measuredAt เป็นเวลาของข้อมูล — ไม่ใช่ sentAt หรือเวลาที่รับ
- ยอมรับ metrics ที่เป็น null รายตัว
- ตีความหน่วยตามสเปค (°C / %RH / µg/m³)

8. การจัดการข้อมูลขาด (Gap Handling)

- ไม่ได้รับข้อมูลบางช่วง = ปกติ ไม่ใช่ error — หมายถึง เส้า / sensor / ระบบต้นทาง down ชั่วคราว
- ใช้ seq + measuredAt ตรวจสอบว่าข้อมูลขาดช่วงใด (เรียงลำดับ / หา gap)
- ผู้ส่งจะไม่ส่งค่าเก่าหลอก — ถ้า sensor ไม่ทำงาน จะ "ไม่ส่ง" (ช่องว่าง ≠ ค่าค้าง)

9. การเสริมความปลอดภัย (Hardening — แนะนำ)

มาตรการ	รายละเอียด	ระดับ
IP allowlist	รับเฉพาะ IP / Cloudflare ranges ของผู้ส่ง (แจ้งให้)	แนะนำ
mTLS	ขอ client certificate ของผู้ส่ง verify เพิ่มอีกชั้น	option
Rate limit + body size limit	กั้น abuse / DoS	แนะนำ
Audit log	log ทุก request (accept/reject + เหตุผล)	แนะนำ

10. สิ่งที่เราส่งมอบ / สิ่งที่ได้รับไม่ต้องทำ

ผู้ส่ง (Smart Pole) ส่งมอบให้

1. Public key (Ed25519) + Key-Id (ผ่านช่องทางปลอดภัย)
2. เอกสารสเปคนี้ + ตัวอย่าง payload / headers
3. Reference receiver code (ตัวอย่างที่ verify ครบ)
4. การทดสอบร่วม: ยิง signed request จริง + ชุด test cases

ผู้รับ ไม่ต้อง ทำ (เพราะเป็น one-way)

1. ไม่ต้องเรียก/ยิงกลับหาผู้ส่ง
2. ไม่ต้องเก็บ secret ของผู้ส่ง — มีแค่ **public key** (เปิดเผยได้ ปลอมข้อความไม่ได้)
3. ไม่ต้อง poll / ดึงข้อมูลเอง

11. ภาคผนวก

A. ตัวอย่าง signed request (curl)

```
curl -X POST https://receiver.example.go.th/smart-pole/ingest \
  -H "Content-Type: application/json" \
  -H "X-SmartPole-Key-Id: spole-2026" \
  -H "X-SmartPole-Timestamp: 1785807300" \
  -H "X-SmartPole-Signature: ed25519=Base64Signature==" \
  -H "X-SmartPole-Event-Id: 9f1c2b7e-4d3a-4b6e-9c11-8a5e2f0d7b34" \
  --data-binary '{"schemaVersion":"1.0","source":"smart-pole",...}'
```

B. Pseudo-code การ verify (ฝั่งผู้รับ)

```
function handlePush(request):
  rawBody    = request.readRawBytes()           # ก่อน parse
  keyId      = request.header("X-SmartPole-Key-Id")
  timestamp  = request.header("X-SmartPole-Timestamp")
  sigB64     = request.header("X-SmartPole-Signature").removePrefix("ed25519=")
  eventId    = request.header("X-SmartPole-Event-Id")

  if abs(now() - timestamp) > 300: return 401    # replay
  pubKey     = lookupPublicKey(keyId)
  message    = timestamp + "." + rawBody
  if not ed25519_verify(pubKey, base64decode(sigB64), message):
    return 401                                # bad signature
  if seenBefore(eventId): return 200            # idempotent
  data       = parseAndValidate(rawBody)       # schema
  store(eventId, data)                         # ใช้ measuredAt
  return 200
```

C. Test cases ที่ receiver ต้องผ่าน

กรณีทดสอบ	ผู้รับต้อง
signed ถูกต้อง	✅ ตอบ 2xx + บันทึกข้อมูล
แก้ body (tamper)	❌ ปฏิเสธ 401 (signature ไม่ตรง)
timestamp เก่ากว่า 5 นาที	❌ ปฏิเสธ 401 (replay)
eventId ซ้ำ	▶️ ตอบ 2xx แต่ไม่บันทึกซ้ำ (dedup)
schema/หน่วยผิด	❌ ปฏิเสธ 4xx
metrics บางตัวเป็น null	✅ รับได้ (บันทึกเท่าที่มี)